FPT UNIVERSITY / EMBEDDED LINUX SYSTEMS

BÁO CÁO THỰC HÀNH

LAB–01

CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG NHÚNG

Hành trình boot từ U-Boot tới BusyBox trên bo ảo Cortex-A9

THÀNH VIÊN THỰC HIỆN

HỌ VÀ TÊN	MÃ SỐ SINH VIÊN
Lê Tiến Thành	SE201799
Lê Tấn Lộc	SE201412
Nguyễn Quang Huy Đạt	SE201415
Trần Đình Khôi Nguyên	SE200558
Trần Trương Tuấn Vũ	SE196467

Nền tảng: Ubuntu (HOST) • ARMv7 / QEMU vexpress-a9 (TARGET)

Thực nghiệm: 09–10/2026 | Tổng hợp: 21/10/2026

1. Mục tiêu và phạm vi triển khai

Bài thực hành tổ chức một hệ Linux ARMv7 tối giản trên QEMU Versatile Express. Nhóm chuẩn bị kernel, chương trình khởi động U-Boot và hệ tập tin gốc dựa trên BusyBox, sau đó kiểm tra từng mắt xích bằng log thực thi thực tế.

Lớp hệ thống	Thành phần / kết quả của nhóm
Máy phát triển	Ubuntu x86_64; cross toolchain arm-linux-gnueabihf-
Máy mô phỏng	QEMU -M vexpress-a9, ARM Cortex-A9, RAM 512 MiB
Firmware/bootloader	U-Boot 2022.04; console serial, môi trường bootargs/bootcmd
Hệ điều hành	Linux 5.15.0; zImage và DTB vexpress-v2p-ca9.dtb
Không gian người dùng	BusyBox 1.35.0 liên kết tĩnh, /sbin/init, /bin/sh
Đầu ra đã kiểm chứng	zImage, DTB, initramfs.cpio.gz và BusyBox shell trên QEMU

Phạm vi bằng chứng: ảnh chụp build/menuconfig thuộc quá trình làm Lab 1; log kiểm thử lại trên bản clone GitHub ngày 21/10/2026 được dùng để xác nhận khả năng boot. Hai đường thử được phân biệt rõ: QEMU nạp thẳng zImage và U-Boot nạp zImage bằng bootz.

2. Mô hình chuỗi khởi động



Hình 1. Sơ đồ do nhóm biên soạn lại: thành phần thực thi, nhiệm vụ và dữ liệu bàn giao trong sáu pha.

Mỗi lần bàn giao đều có đối tượng cụ thể: firmware chuyển sang U-Boot; U-Boot cung cấp zImage, DTB, initramfs và tham số khởi động; Linux trao quyền cho tiến trình init; BusyBox init mở shell sau khi chạy script rcS. Trong môi trường QEMU, giai đoạn ROM/đặt lại phần cứng là mô phỏng, không có một ROM vật lý của máy thật được đo trực tiếp.

2.1 Phân biệt hai chế độ boot kiểm chứng

Chế độ	Ai nạp kernel?	Giá trị kiểm tra
Direct boot	QEMU: -kernel zImage, -dtb, -initrd	Kernel + initramfs tự chạy tới shell

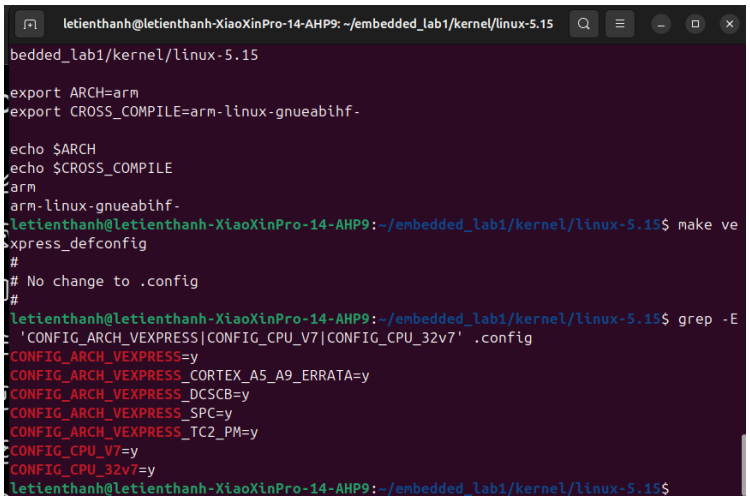
Chế độ	Ai nạp kernel?	Giá trị kiểm tra
Boot thông qua U-Boot	U-Boot: bootz <kernel> <initrd:size> <fdt>	Kiểm tra được cả bootloader và chuyển quyền sang Linux

3. Phần A — Biên dịch và cấu hình Linux Kernel

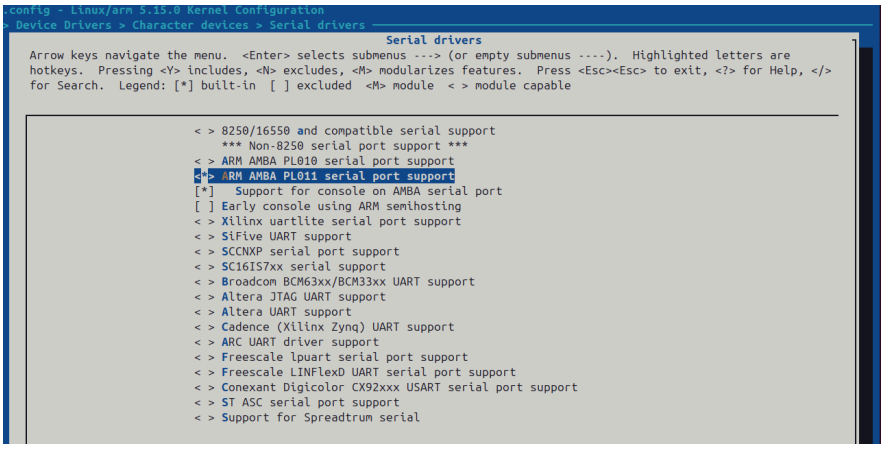
3.1 Lựa chọn target và tập cấu hình

Target của lab là Versatile Express Cortex-A9. Nhóm sử dụng vexpress_defconfig cho ARCH=arm và biên dịch chéo bằng arm-linux-gnueabihf-. Đây là lựa chọn phù hợp hơn versatile_defconfig vốn dành cho dòng Versatile khác. Trước khi build, nhóm đối chiếu các option ARMv7, Versatile Express, PL011 console, initramfs và SMP.

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_defconfig
make -j$(nproc) zImage dtbs modules
```



Hình 2. Ảnh nhóm: thiết lập kiến trúc ARM và kiểm tra option của kernel.

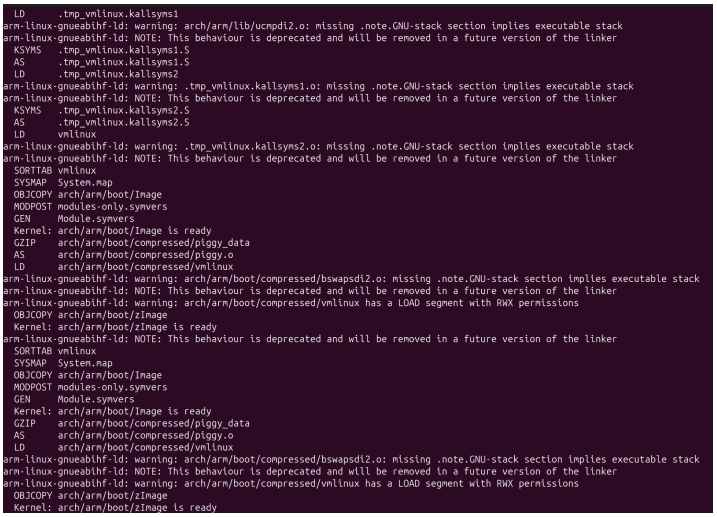


Hình 3. Ảnh nhóm: serial PL011 được cấu hình cho console ttyAMA0.

3.2 Kết quả tạo binary ARM

Sau giai đoạn make, output có Linux zImage và Device Tree Blob cho vexpress-v2p-ca9. Khi kiểm tra trên bản clone, lệnh file báo “Linux kernel ARM boot executable zImage (little-endian)”, còn các tùy chọn

CONFIG_ARCH_VEXPRESS, CONFIG_BLK_DEV_INITRD, CONFIG_DEVTMPFS, CONFIG_SERIAL_AMBA_PL011 và CONFIG_SMP đều bật trong kernel.config.



Hình 4. Đoạn cuối build kernel; các image cần thiết được tạo.



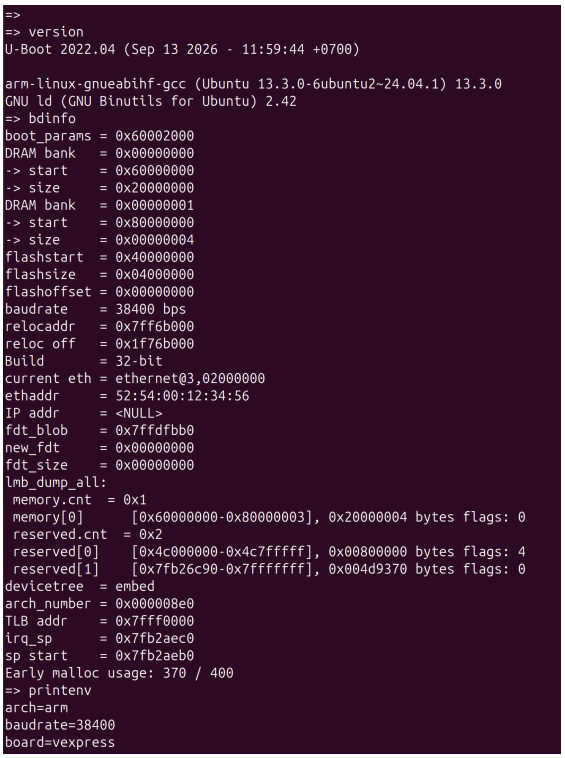
Hình 5. Xác nhận vị trí, định dạng và kích thước của zImage.

Kết luận Phần A: binary đúng họ ARM, có cấu hình đáp ứng nền tảng vexpress-a9 và đã được khởi động thực tế. Các dòng cảnh báo build, nếu có, cần đọc theo mức độ ảnh hưởng thay vì coi tất cả đều là lỗi dừng build.

4. Phần B — U-Boot và tham số khởi động

4.1 Môi trường và chức năng của bootloader

U-Boot 2022.04 đã hiện prompt => trên QEMU, nhận DRAM 512 MiB, serial console và Ethernet. Nhóm dùng version, bdinfo, printenv để đọc trạng thái ban đầu. Ở lần khởi động U-Boot, thông báo “bad CRC, using default environment” có nghĩa flash environment không hợp lệ nên dùng giá trị mặc định; nó không cản trở việc vào shell.



Hình 6. Log thực hành của nhóm: U-Boot shell, thông tin bo và các biến môi trường.

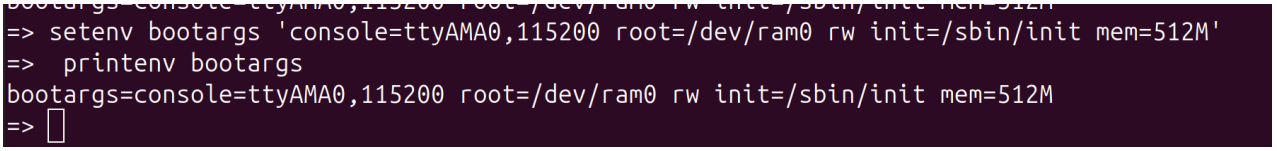
4.2 bootargs, bootcmd và vị trí image trong RAM

Các tham số được truyền sang Linux qua /chosen trong DTB. Việc gán bootargs trước khi boot cho phép chọn console ttyAMA0 và tiến trình init. Mỗi địa chỉ bootz phải trùng nơi image đã thực sự được nạp; initramfs dạng raw cần truyền chính xác kích thước sau dấu hai chấm.

```
# Lệnh đã được thực nghiệm thành công trên bản clone
setenv bootargs 'console=ttyAMA0,115200 rdinit=/sbin/init'
setenv bootcmd 'bootz 0x62000000 0x64000000:0x1a0000 0x66000000'
run bootcmd
```

Trong bài thử ngày 21/10, ba image được QEMU đặt trước vào RAM tại 0x62000000 (zImage), 0x64000000 (initramfs), 0x66000000 (DTB). U-Boot đọc được DTB magic 0xd00dfeed; bootz in “Starting kernel ...”; Linux tiếp tục chạy tới BusyBox shell. Giá trị initramfs 0x1a0000 được dùng trong lần chạy thành công, nhưng cần đối chiếu kích thước thật bằng stat trước khi dùng như một công thức tái lập cho bản image khác.

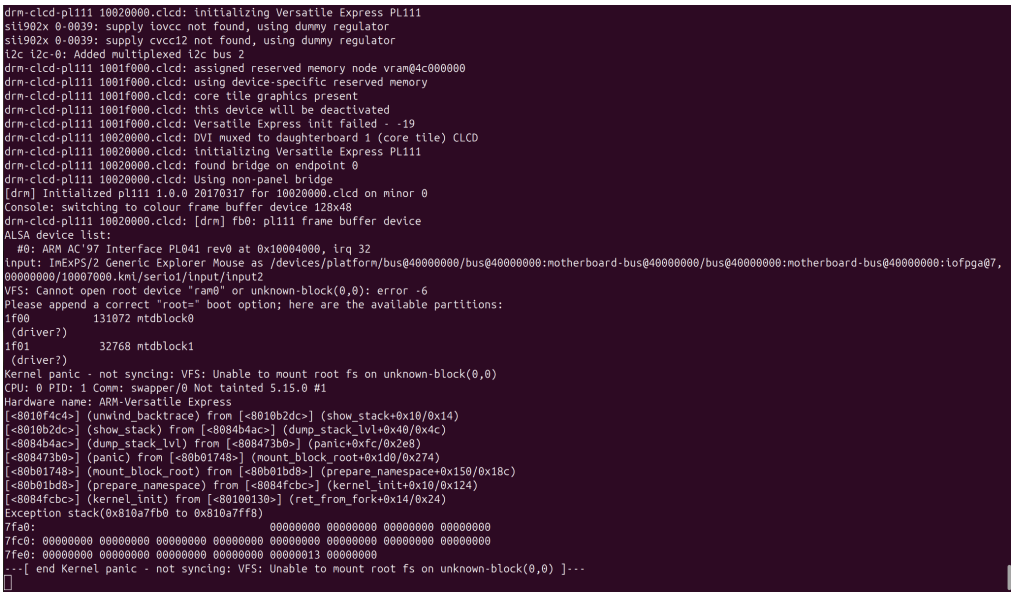
Ở lần thử với DTB tại 0x60000000, U-Boot báo FDT_ERR_BADSTRUCTURE và reset. Nhóm đã kiểm tra lại header DTB, bố trí tách các vùng nạp trong RAM rồi thử lại thành công. Đây là một kết quả chẩn đoán, không đồng nghĩa DTB nguồn bị hỏng.



Hình 7. Ảnh nhóm: đặt bootargs và dùng printenv để xác minh.

4.3 Thử nghiệm lỗi khi không có root filesystem

Khi kernel được yêu cầu mount rootfs nhưng không có initramfs phù hợp, quá trình dừng ở bước mount root, thường kèm “VFS: Unable to mount root fs” hoặc kernel panic. Điều này giải thích vì sao zImage hoạt động không đồng nghĩa hệ thống đã có userspace.

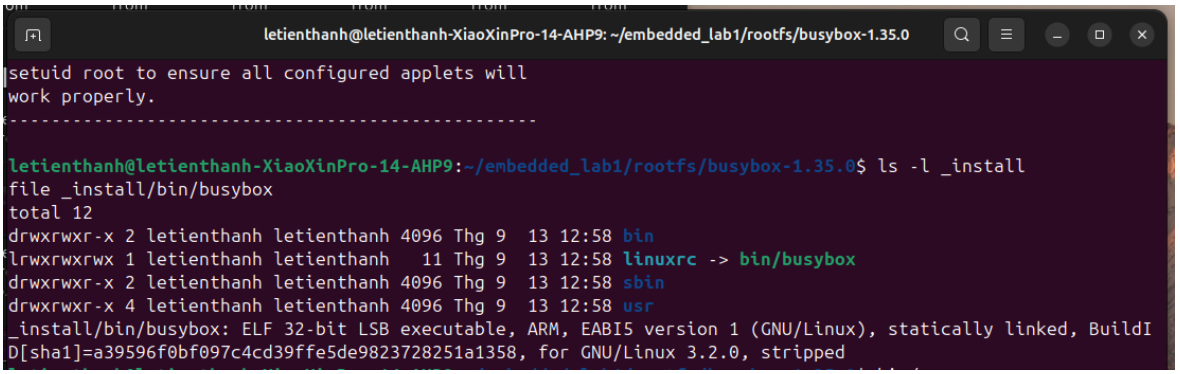


Hình 8. Ảnh từ quá trình làm bài: kernel panic trong tình huống thiếu rootfs.

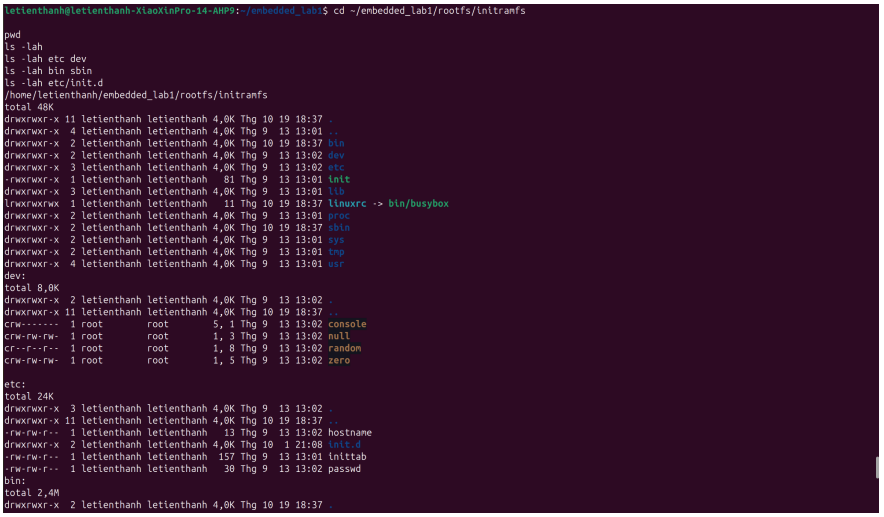
5. Phần C — BusyBox, initramfs và xác nhận hệ thống

5.1 Chuẩn bị môi trường người dùng

BusyBox được build cho ARM EABI5 ở chế độ statically linked. Với cách này, initramfs không phụ thuộc dynamic linker hoặc thư viện chia sẻ từ host. Nhóm bố trí /bin, /sbin, /etc, /proc, /sys, /tmp cùng các device node cần thiết và tạo file initramfs.cpio.gz bằng định dạng cpio newc.



Hình 9. File BusyBox ARM có kiểu static ELF32.

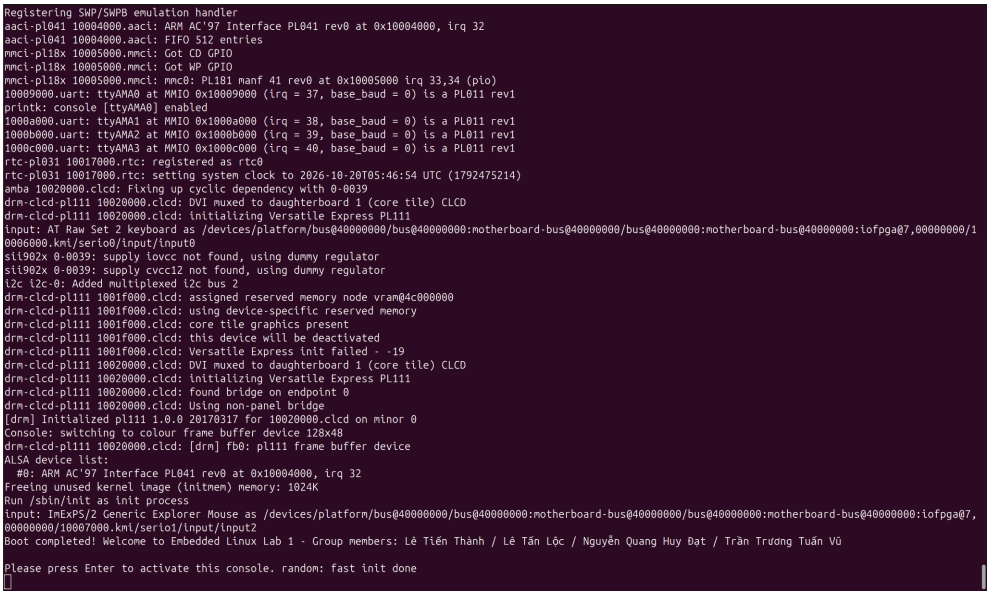


Hình 10. Các thư mục và device node phục vụ initramfs tối giản.

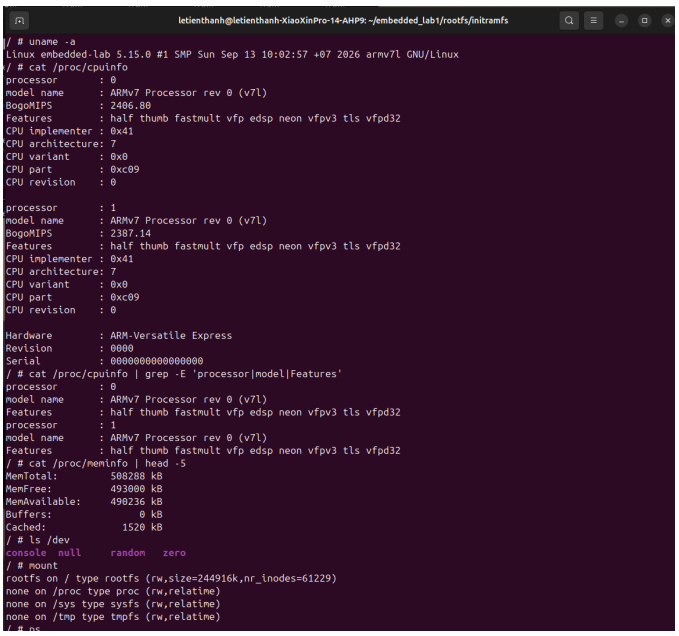
```
# Kiểm tra gói rootfs sau khi dựng
file rootfs/initramfs/bin/busybox
ls -lh output/initramfs.cpio.gz
# Bản thực nghiệm dùng cpio định dạng newc, sau đó gzip
```

5.2 Khởi động tới shell và kiểm chứng tại runtime

Lần chạy QEMU trực tiếp và lần chạy U-Boot → Kernel đều in dòng chào của rcS, sau đó mở shell /. Các lệnh uname -a, ps, mount và /proc/cmdline giúp xác nhận không chỉ boot log mà cả trạng thái sống của Linux.



Hình 11. Ảnh nhóm: boot tới console BusyBox trên QEMU.



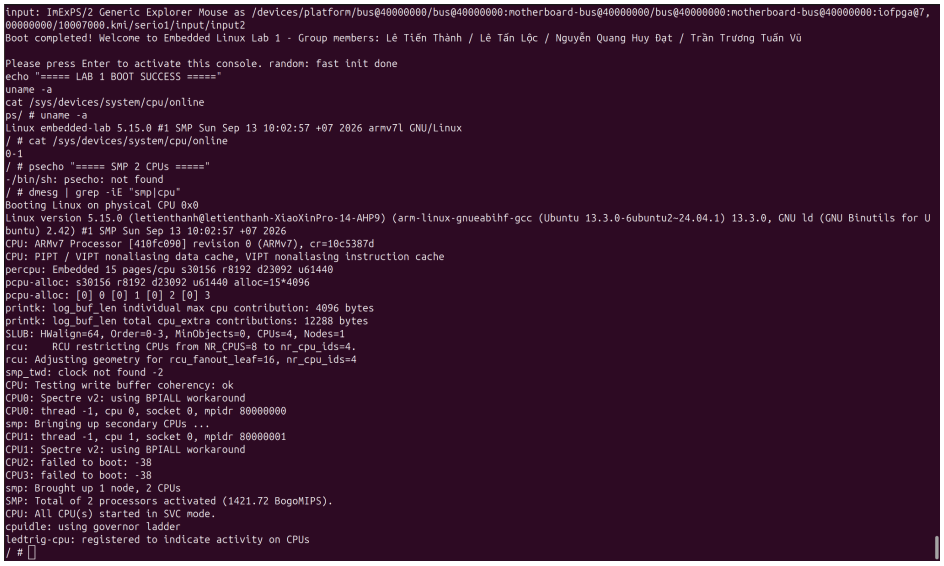
Hình 12. Kiểm tra phiên bản Linux ARMv7, /proc/cpuinfo và các filesystem đã mount.

Lệnh / quan sát	Dấu hiệu hoạt động
uname -a	Linux embedded-lab 5.15.0 ... armv7l GNU/Linux
ps	init mang PID 1; có shell /bin/sh và kernel threads
mount	rootfs trên /; proc, sysfs và tmpfs /tmp có mặt

Lệnh / quan sát	Dấu hiệu hoạt động
cat /proc/cmdline	console=ttyAMA0,115200 rdinit=/sbin/init ở lần boot qua U-Boot
cat /proc/cpuinfo	CPU implementer 0x41; part 0xc09 (Cortex-A9)

5.3 SMP trên máy ảo

Build có CONFIG_SMP=y nhưng số CPU online thay đổi theo tham số -smp của QEMU và cách bring-up secondary CPU. Bản direct boot trước có thread theo nhiều lõi; riêng bản boot qua U-Boot ngày 21/10 log ghi “SMP: Total of 1 processors activated”. Không nên suy ra số CPU đang hoạt động chỉ từ CONFIG_SMP hoặc từ các dòng cpuhp/N trong ps.



Hình 13. Minh chứng riêng của nhóm khi so sánh thông tin SMP trong quá trình boot.

6. Trả lời câu hỏi phân tích

Câu 1 Phân biệt gnueabi và gnueabihf

Sự khác nhau trọng tâm là ABI truyền tham số số thực. Với soft-float ABI, đối số float/double không tuân theo quy ước đưa vào thanh ghi VFP; với hard-float ABI, chương trình có thể truyền tham số số thực qua các thanh ghi VFP theo chuẩn tương ứng. User-space ARMv7 có VFP của Cortex-A9 nên gnueabihf là phù hợp. Riêng kernel Linux không sử dụng FPU như chương trình userspace và cần flags build riêng; không nên áp dụng tùy tiện -mfloat-abi=hard cho kernel.

Câu 2 Vì sao bộ nhớ vexpress-a9 bắt đầu tại 0x60000000?

Versatile Express CoreTile Cortex-A9 đặt DRAM vào vùng địa chỉ cao hơn so với bo Versatile PB. Các vùng địa chỉ thấp được dành cho flash và ngoại vi, vì vậy image kernel, initramfs và DTB phải được nạp vào khoảng RAM mà bo thực sự cung cấp. Một địa chỉ bootz nằm ngoài RAM, hoặc các vùng image đè lên nhau, có thể khiến quá trình chuyển quyền từ U-Boot thất bại.

Câu 3 Phân tích giá trị MIDR 0x410FC090

Bit	Trường	Ý nghĩa
31–24	Implementer = 0x41	ARM Ltd.
23–20	Variant = 0x0	Revision lớn r0
19–16	Architecture = 0xF	Thông tin kiến trúc qua ID registers
15–4	Part = 0xC09	Cortex-A9

Bit	Trường	Ý nghĩa
3–0	Revision = 0x0	Bản p0

Câu 4 Vì sao chọn BusyBox static?

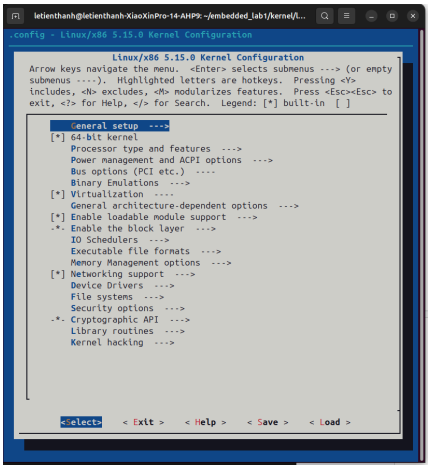
Một initramfs tối thiểu không chứa thư viện .so đầy đủ và dynamic loader. BusyBox liên kết tĩnh có thể được chạy trực tiếp trong giai đoạn /sbin/init, giúp giảm nguy cơ thiếu ld-linux-armhf.so.3 hoặc libc.so.6 dẫn tới không chạy được init. Nếu chọn dynamic, phải truy vết chính xác DT_NEEDED và copy các dependency thích hợp cho ARM hard-float.

Câu 5 SMP ảnh hưởng thế nào tới boot và các thread?

SMP bổ sung bước đánh thức CPU phụ, khởi tạo per-CPU data và scheduler runqueue trên từng CPU online. Khi nhiều CPU được kích hoạt, log có các dòng bring-up CPU bổ sung và ps thường có nhóm thread migration/N, ksoftirqd/N, cpuhp/N. Phải phân biệt possible/present/online với tham số -smp và quan sát boot thực tế, không dựa hoàn toàn vào số mục thread.

7. Khó khăn, xử lý và bài học

Tình huống	Chẩn đoán	Cách xử lý
menuconfig chạy ra x86	Thiếu ARCH=arm khi cấu hình	Thiết lập ARCH, CROSS_COMPILE trước make
Cấu hình thiếu CPU_V7	Kết quả config chưa đúng mục tiêu	Nạp lại vexpress_defconfig, grep kiểm tra trước build
U-Boot báo FDT_ERR_BADSTRUCTURE	DTB bị xử lý sai trong sơ đồ nạp RAM của lần thử	Dời DTB sang vùng RAM tách biệt; xác minh fdt header
Boot có Kernel nhưng thiếu rootfs	Không có initramfs phù hợp khi mount root	Tạo initramfs từ BusyBox static, chỉ rõ địa chỉ và kích thước khi bootz



Hình 14. Chẩn đoán ban đầu: menuconfig hiển thị kiến trúc x86.

8. Đối chiếu kết quả và kết luận

Hạng mục	Minh chứng trong báo cáo	Đánh giá
Build và cấu hình ARMv7	Binary zImage ARM; kernel.config và log compile	Đạt
U-Boot hoạt động	Prompt =>; version, bdinfo, printenv; bootz	Đạt
Đến userspace	Kernel /sbin/init; shell BusyBox; uname/mount/ps	Đạt
Rootfs bị thiếu	Có ảnh kernel panic của giai đoạn không nạp initramfs	Có minh chứng

Hạng mục	Minh chứng trong báo cáo	Đánh giá
Hồ sơ nộp	Báo cáo này cần được xuất PDF và đưa vào bao_cao/ trên GitHub	Cần nộp

Kết luận: toàn bộ mắt xích chính của Linux nhúng đã được chứng minh bằng thử nghiệm trên ARMv7 giả lập. Việc clone lại repository và khởi chạy cả đường QEMU direct boot lẫn U-Boot bootz cung cấp căn cứ kiểm chứng độc lập với cây build ban đầu. Báo cáo không xem các cảnh báo phụ của thiết bị đồ họa là lỗi fatal vì hệ thống vẫn vào console.

Tài liệu đối chiếu: Đề LAB-01 môn Embedded Linux Systems; tài liệu Lab01_Report.pdf do người dùng cung cấp (tham khảo phạm vi và nguyên lý, không sử dụng hình hoặc log của nhóm khác); bộ ảnh/log thực hành Lab 1 của nhóm; log clone và boot ngày 21/10/2026.